

Part 2 — Design, resampling and preparation

Complete tutorial for sciModelFlowR (version 1.0.2)

Walter Esfrain Pereira

2026-09-19

Table of contents

1	Why resampling decides the result	3
2	Resampling schemes	4
3	The price of leakage, measured	7
4	Bootstrap: the right unit changes the interval	10
5	Preparation: imputation, centring and scaling	13
6	Imbalance: touch training only	16
7	Feature selection and representation	18
8	Part 2 exercises	24

1 Why resampling decides the result

The scientific question is always “how does this model behave on new observations?”. The answer depends entirely on **what counts as a new observation**. If plots from the same farm appear in training and testing, the model memorises the farm and the estimate becomes optimistic. If preprocessing is fitted on the whole dataset before splitting, the test median leaks into training. `sciModelFlowR` treats resampling as part of the **design**: `smf_design_spec()` declares groups, blocks, time and coordinates, and the package refuses incompatible schemes instead of warning.

2 Resampling schemes

Each simulated dataset has a structure that calls for its own scheme. The table shows what was declared and what was built.

```
field <- ler("field_yield")
farms <- ler("farm_groups")
climate <- ler("climate_series")
soil <- ler("soil_spatial")

h <- smf_holdout_split(field, train_prop = 0.7, id_column = "plot_id", seed = 101L)
v <- smf_vfold_cv(field, v = 5L, id_column = "plot_id", seed = 102L)
gv <- smf_group_vfold_cv(farms, groups = "farm", v = 6L, id_column = "plot_id", seed = 103L)
tc <- smf_time_cv(climate, time = "day", initial = 400L, assess = 100L, id_column = "day", seed = 104L)
sc <- smf_spatial_cv(soil, coords = c("x", "y"), v = 4L, id_column = "point_id", seed = 105L)

scheme_summary <- data.frame(
  scheme = c("holdout (70/30)", "v-fold (v = 5)", "group v-fold (farm)",
    "time series CV", "spatial CV"),
  partitions = c(1L, length(v@splits), length(gv@splits), length(tc@splits), length(sc@splits)),
  property = c("one random (stratifiable) split",
    "each unit in the test set exactly once",
    "whole farms out of training",
    "past trains, future tests",
    "whole spatial blocks out of training")
)
tab_smf(scheme_summary, caption = "Resampling schemes built for each data structure.")
```

Table 2.1: Resampling schemes built for each data structure.

scheme	partitions	property
holdout (70/30)	1	one random (stratifiable) split
v-fold (v = 5)	5	each unit in the test set exactly once
group v-fold (farm)	6	whole farms out of training
time series CV	3	past trains, future tests
spatial CV	4	whole spatial blocks out of training

```
units <- 1:12
block <- function(name, folds, role_of) {
  do.call(rbind, lapply(folds, function(f) data.frame(
```

```

    scheme = name, unit = units, fold = f, role = role_of(f)
  )))
}
scheme_df <- rbind(
  block("Holdout", 1L, function(f) ifelse(units <= 8, "train", "test")),
  block("V-fold", 1:3, function(f) ifelse(ceiling(units / 4) == f, "test", "train")),
  block("Group v-fold", 1:3, function(f) ifelse(ceiling(units / 4) == f, "test", "train")),
  block("Time series", 1:3, function(f) ifelse(units > 4 * f & units <= 4 * (f + 1),
                                             "test", ifelse(units <= 4 * f, "train", "out")),
  block("Spatial blocks", 1:3, function(f) ifelse(ceiling(units / 4) == f, "test", "train"))
)
scheme_df$scheme <- factor(scheme_df$scheme, levels = c("Holdout", "V-fold", "Group v-fold",
                                                       "Time series", "Spatial blocks"))
ggplot(scheme_df, aes(unit, factor(fold), fill = role)) +
  geom_tile(colour = "white", linewidth = .8) +
  facet_wrap(~scheme, ncol = 1) +
  scale_fill_manual(values = c(train = cores_smf[["primaria"]],
                                test = cores_smf[["destaque"]],
                                out = "#e8e8e8")) +
  labs(x = "experimental unit", y = "partition", fill = NULL,
       title = "Resampling schemes: what trains and what tests") +
  theme(legend.position = "top", panel.grid = element_blank())

```

Resampling schemes: what trains and what tests

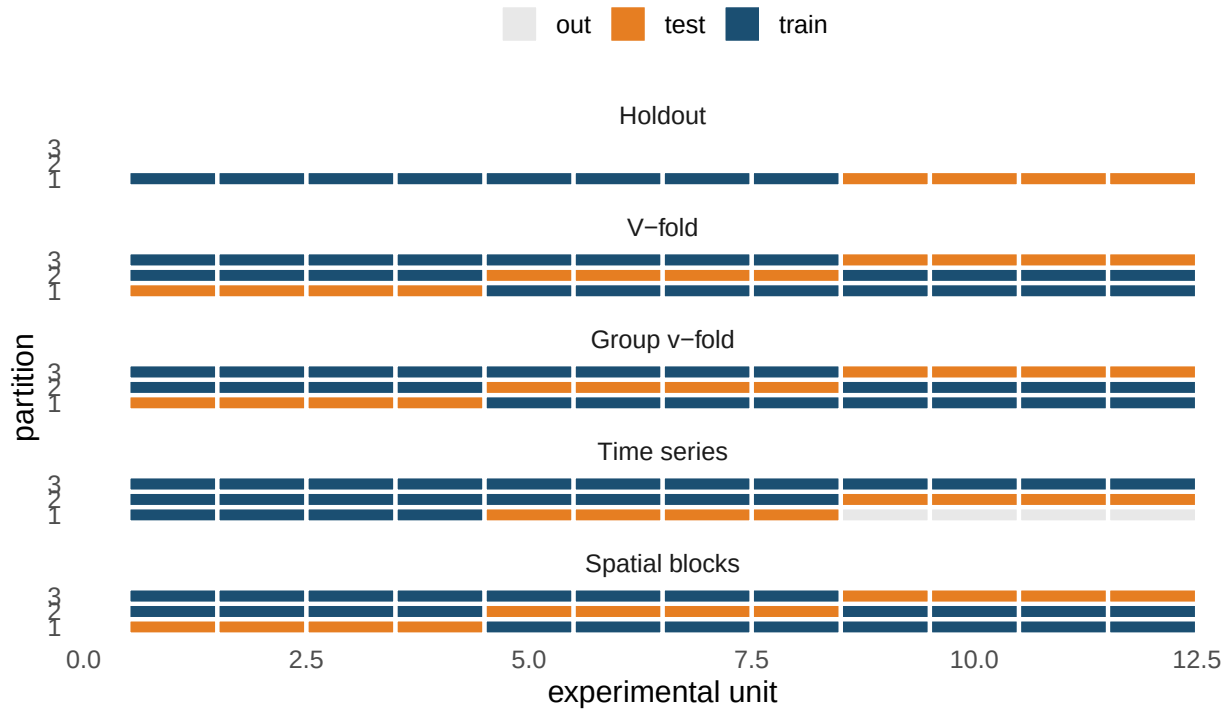


Figure 2.1: How each scheme partitions the data: training in blue, testing in orange. The wrong scheme puts neighbours on both sides of the border.

i Interpretation

In ordinary v-fold, neighbours from the same farm (or the same spatial block) appear on both sides of the border. In the **group**, **time series** and **spatial** schemes, the border respects dependence: the whole farm leaves, the future does not train the past, the whole spatial block is held out. Choosing the scheme **is** a scientific decision — and it must be declared.

3 The price of leakage, measured

The best way to learn is to watch the error happen. The package **refuses** ordinary v-fold when groups are declared:

```
spec_iid <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "farm"), id_column = "plot_id",
    categorical = "farm"),
  design = smf_design_spec(id_column = "plot_id", group_columns = "farm"),
  preprocessing = smf_preprocess_spec("median", center = TRUE, scale = TRUE),
  resampling = smf_resampling_spec("kfold", n_splits = 6L, seed = 11L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
  reproducibility = smf_reproducibility_spec(11L)
)
refusal <- tryCatch(smf_resample_experiment(spec_iid, farms),
  error = function(e) conditionMessage(e))
cat(refusal, "\n")
```

Row-wise random resampling is incompatible with declared grouped/repeated units by default.

To **quantify** the optimism, we force the same scheme with `override = TRUE` and compare with the correct scheme. The model includes the farm as a predictor: with ordinary v-fold, the model learns each farm's effect from the other plots and nails the test set; with **group**, the test farm is unseen.

```
spec_grp <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "farm"), id_column = "plot_id",
    categorical = "farm"),
  design = smf_design_spec(id_column = "plot_id", group_columns = "farm"),
  preprocessing = smf_preprocess_spec("median", center = TRUE, scale = TRUE),
  resampling = smf_resampling_spec("group", n_splits = 6L, seed = 11L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
  reproducibility = smf_reproducibility_spec(11L)
)
res_iid <- smf_resample_experiment(spec_iid, farms, override = TRUE)
res_grp <- smf_resample_experiment(spec_grp, farms)
rmse_iid <- res_iid@aggregate$mean[res_iid@aggregate$metric == "rmse"]
rmse_grp <- res_grp@aggregate$mean[res_grp@aggregate$metric == "rmse"]
```

```
leak_df <- data.frame(
  scheme = c("ordinary v-fold (forced)", "group v-fold (correct)"),
  rmse = c(rmse_iid, rmse_grp)
)
ggplot(leak_df, aes(scheme, rmse, fill = scheme)) +
  geom_col(width = .55, show.legend = FALSE) +
  geom_hline(yintercept = verdade$farm_groups$residual_sd, linetype = "dashed",
    colour = cores_smf[["sucesso"]]) +
  geom_hline(yintercept = sqrt(verdade$farm_groups$residual_sd^2 +
    verdade$farm_groups$farm_sd^2),
    linetype = "dotted", colour = cores_smf[["alerta"]]) +
  geom_text(aes(label = fmt(rmse, 1)), vjust = -0.4) +
  scale_fill_manual(values = c(cores_smf[["alerta"]], cores_smf[["primaria"]])) +
  annotate("text", x = 2.45, y = verdade$farm_groups$residual_sd, hjust = 1, vjust = -0.4,
    label = "residual (120)", colour = cores_smf[["sucesso"]], size = 3.2) +
  annotate("text", x = 2.45, y = sqrt(verdade$farm_groups$residual_sd^2 +
    verdade$farm_groups$farm_sd^2), hjust = 1, vjust = -0.4,
    label = "residual + farm (323)", colour = cores_smf[["alerta"]], size = 3.2) +
  labs(x = NULL, y = "RMSE (kg/ha)",
    title = "Leakage costs: the same model, two resampling schemes") +
  ylim(0, 360)
```

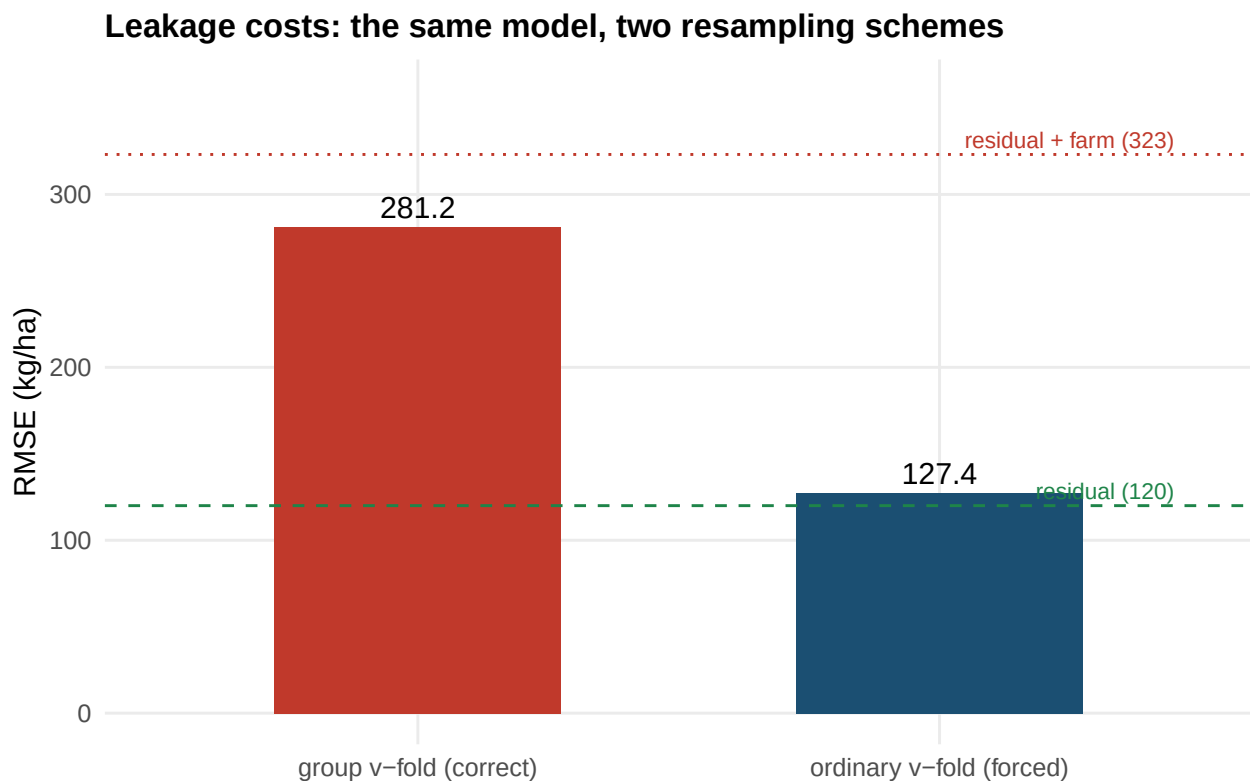


Figure 3.1: The same model, two schemes: forced ordinary v-fold returns a much smaller RMSE than group v-fold. Lines mark the simulated truth.

! Interpretation

With the forced scheme, RMSE drops to 127.4 kg/ha — below even the true residual spread (120). That is impossible: the number is optimistic by construction, because the model saw the same farm on both sides. With **group**, RMSE rises to 281.2, compatible with the sum of the two variance sources (323). The package refused the wrong path; **override** exists to **study** the problem, not to use it in production.

4 Bootstrap: the right unit changes the interval

The bootstrap answers “how much does this estimate vary if the experiment were repeated?”. The answer is only honest if the resampling respects the unit that varies between repetitions. In the farm trial, the unit is the **farm**, not the plot.

```
b_case <- tryCatch(
  smf_bootstrap(farms, smf_bootstrap_spec("case", n_resamples = 99L,
                                           interval = "percentile", seed = 7L),
    statistic = function(z) c(mean = mean(z$yield)),
    design = smf_design_spec(id_column = "plot_id",
                             group_columns = "farm")),
  error = function(e) conditionMessage(e))
b_bca_cl <- tryCatch(
  smf_bootstrap(farms, smf_bootstrap_spec("cluster", n_resamples = 99L,
                                           sampling_unit = "farm",
                                           interval = "bca", seed = 7L),
    statistic = function(z) c(mean = mean(z$yield)),
    design = smf_design_spec(id_column = "plot_id",
                             group_columns = "farm")),
  error = function(e) conditionMessage(e))
cat("case (iid):", b_case, "\n\n")
```

case (iid): The requested bootstrap unit does not preserve dependence declared in DesignSpec.

```
cat("BCa with cluster:", b_bca_cl, "\n")
```

BCa with cluster: The generic row-delete BCa implementation is available only for case bootstrap aware schemes require a scheme-specific jackknife and are blocked rather than approximated.

```
b_cl <- smf_bootstrap(farms,
  smf_bootstrap_spec("cluster", n_resamples = 1999L, sampling_unit = "farm",
    interval = "percentile", seed = 7L),
  statistic = function(z) c(mean = mean(z$yield)),
  design = smf_design_spec(id_column = "plot_id", group_columns = "farm"))
ci_cl <- smf_bootstrap_interval(b_cl)
stability <- smf_bootstrap_stability(b_cl)
```

```
p1 <- ggplot(data.frame(mean = b_cl@estimates[[1]]), aes(mean)) +
  geom_histogram(fill = cores_smf[["primaria"]], colour = "white", bins = 35) +
  geom_vline(xintercept = ci_cl["mean", ], linetype = "dashed",
             colour = cores_smf[["alerta"]]) +
  labs(x = "mean yield (kg/ha)", y = "resamples",
       title = "Cluster bootstrap distribution",
       subtitle = paste0("95% percentile CI: [", fmt(ci_cl["mean", 1], 0),
                        "; ", fmt(ci_cl["mean", 2], 0), "]"))
p2 <- ggplot(subset(stability, statistic == "mean"), aes(B, width)) +
  geom_line(colour = cores_smf[["primaria"]]) +
  geom_point(size = 1.4) +
  labs(x = "resamples B", y = "95% CI width",
       title = "Interval width stabilizes with B")
patchwork::wrap_plots(p1, p2)
```

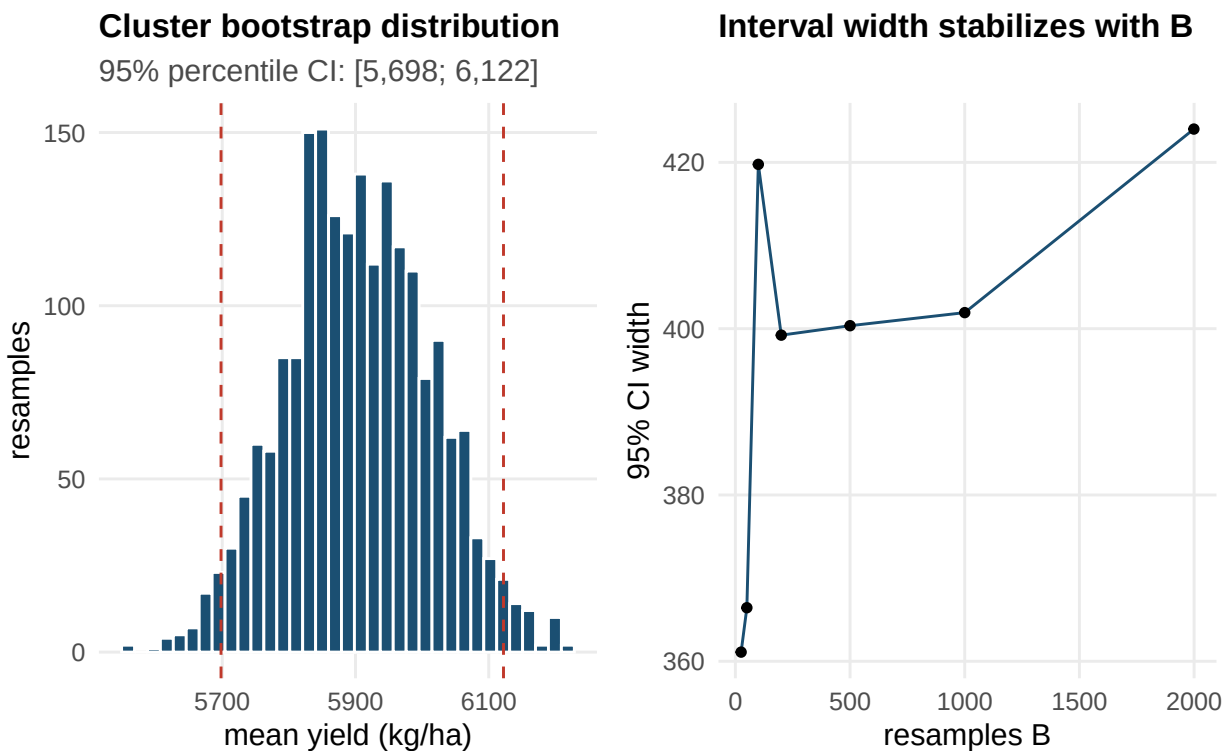


Figure 4.1: Bootstrap distribution of the mean (unit = farm) and interval width as B grows: with 1999 resamples the width has stabilised.

⚠ Trap

The **case** bootstrap (plot by plot) was **refused** because it would underestimate variability: plots from the same farm are not independent repetitions. BCa with cluster was also refused — bias correction would need a scheme-specific jackknife, and the package prefers to block rather

than approximate. Both blocks are the same philosophy: **an explicit error is better than a plausible, wrong number.**

5 Preparation: imputation, centring and scaling

Part 1 showed two kinds of missingness: MCAR in nitrogen and MAR (block B4) in organic matter. Before comparing imputation strategies, however, one mandatory step: **the outliers**. The table below shows the organic-matter coefficient (planted truth: 120) in four scenarios.

```
no_out <- field[!field$plot_id %in% verdade$field_yield$outlier_ids, ]
dspec <- smf_data_spec("yield", c("nitrogen", "water", "organic_matter"),
  id_column = "plot_id")
om_coef <- function(data, imp) {
  pp <- smf_fit_preprocessor(data, smf_preprocess_spec(imp), dspec, role = "training")
  tr <- smf_apply_preprocessor(pp, data)
  tr[["yield"]] <- data[["yield"]]
  fit <- lm(yield ~ nitrogen + water + organic_matter, data = tr)
  c(slope = unname(coef(fit)[["organic_matter"]]),
    n = sum(stats::complete.cases(tr)))
}
c_all <- om_coef(field, "none")
c_none <- om_coef(no_out, "none")
c_mean <- om_coef(no_out, "mean")
c_med <- om_coef(no_out, "median")
imp_tbl <- data.frame(
  scenario = c("all data, no imputation",
    "no outliers, no imputation (complete cases)",
    "no outliers, mean imputation",
    "no outliers, median imputation"),
  om_coef = c(c_all["slope"], c_none["slope"], c_mean["slope"], c_med["slope"]),
  rows_used = c(c_all["n"], c_none["n"], c_mean["n"], c_med["n"]))
)
tab_smf(imp_tbl, caption = "Organic-matter coefficient under different discrepancy-handling c
```

Table 5.1: Organic-matter coefficient under different discrepancy-handling decisions (planted truth: 120).

scenario	om_coef	rows_used
all data, no imputation	285.808	153
no outliers, no imputation (complete cases)	115.286	149
no outliers, mean imputation	198.395	175
no outliers, median imputation	206.427	175

```
ggplot(imp_tbl, aes(stats::reorder(scenario, om_coef), om_coef)) +
  geom_col(fill = cores_smf[["primaria"]], width = .6) +
  geom_hline(yintercept = verdade$field_yield$beta[["organic_matter"]],
             linetype = "dashed", colour = cores_smf[["alerta"]]) +
  geom_text(aes(label = fmt(om_coef, 1)), hjust = -0.15) +
  coord_flip() +
  labs(x = NULL, y = "estimated coefficient (kg/ha per % OM)",
       title = "Outliers first, imputation second") +
  ylim(0, max(imp_tbl$om_coef) * 1.15)
```

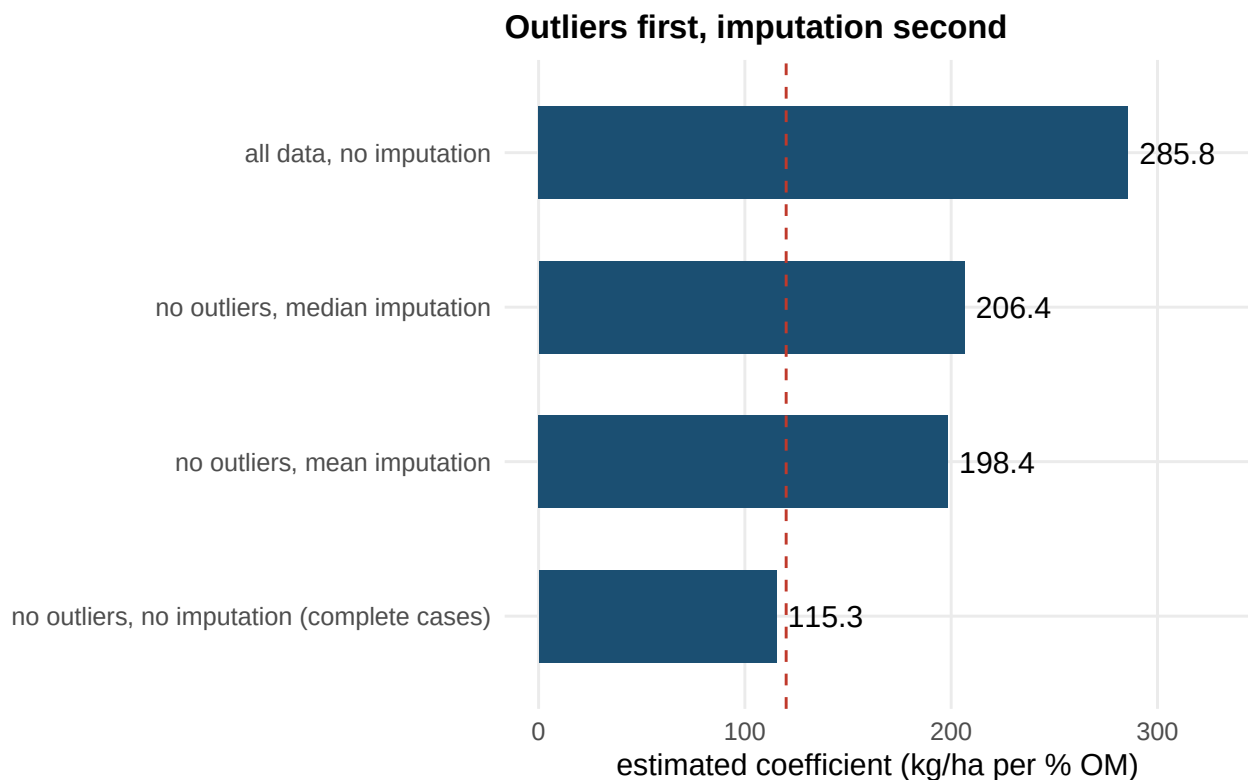


Figure 5.1: The organic-matter coefficient: outliers dominate the error; once removed, the imputation choice still shifts the estimate.

! Interpretation

Three lessons in one table. (1) With the five outliers in, the coefficient almost triples (286 against the truth 120) — numerical discrepancies are the first problem, not missingness. (2) With the outliers removed, complete cases recover 115.3, close to the truth, but use only 149 of the 180 plots. (3) Mean or median imputation uses all rows, but shifts the coefficient to 198/206: imputing is **not neutral**. The decision must be declared and stress-tested.

```
coef_scale <- function(center, scale) {
  pp <- smf_fit_preprocessor(no_out, smf_preprocess_spec("median", center = center,
```

```

                                scale = scale),
                                dspec, role = "training")
tr <- smf_apply_preprocessor(pp, no_out)
tr[["yield"]] <- no_out[["yield"]]
fit <- lm(yield ~ nitrogen + water + organic_matter, data = tr)
round(coef(fit)[-1], 3)
}
scale_tbl <- rbind(
  "no centre, no scale" = coef_scale(FALSE, FALSE),
  "centred" = coef_scale(TRUE, FALSE),
  "centred and scaled" = coef_scale(TRUE, TRUE)
)
tab_smf(as.data.frame(scale_tbl), caption = "Effect of centring and scaling on coefficients:

```

Table 5.2: Effect of centring and scaling on coefficients: centring does not change slopes; scaling changes the unit.

	nitrogen	water	organic_matter
no centre, no scale	24.004	2.744	206.427
centred	24.004	2.744	206.427
centred and scaled	1044.576	302.382	91.741

💡 Good practice

Centring does not change slopes (it only shifts the intercept). Scaling changes the unit of the coefficient — and is essential for comparing the importance of predictors on different scales. The package fits preprocessing **inside each fold**; in Parts 3 and 4 this happens automatically inside `smf_fit_experiment()` and `smf_resample_experiment()`.

6 Imbalance: touch training only

```
disease <- ler("disease_binary")
disease$disease <- factor(disease$disease, levels = c(0, 1))
```

In the disease dataset, the positive class is 13.8% and label noise caps the ceiling. Balancing strategies resample **training**; testing stays untouched.

```
balance_tbl <- do.call(rbind, lapply(c("none", "downsample", "upsample", "smote"), function(m) {
  out <- tryCatch(smf_apply_imbalance(disease, "disease", smf_imbalance_spec(m),
                                     context = "training", seed = 20260919),
                 error = function(e) conditionMessage(e))
  if (is.character(out)) {
    data.frame(method = m, class_0 = NA_integer_, class_1 = NA_integer_,
              note = out)
  } else {
    data.frame(method = m,
              class_0 = sum(out@data$disease == "0"),
              class_1 = sum(out@data$disease == "1"),
              note = "")
  }
}))
tab_smf(balance_tbl, caption = "Class counts before and after each balancing strategy (training only)")
```

Table 6.1: Class counts before and after each balancing strategy (training only).

method	class_0	class_1	note
none	345	55	
down- sam- ple	55	55	
up- sam- ple	345	345	
smote	NA	NA	Error in step_smote():

Caused by error in prep(): x All columns selected for the step should be double or integer. * 1 factor variable found: plant_id |

```

balance_long <- data.frame(
  method = rep(balance_tbl$method, 2),
  class = rep(c("healthy (0)", "diseased (1)"), each = nrow(balance_tbl)),
  n = c(balance_tbl$class_0, balance_tbl$class_1)
)
ggplot(subset(balance_long, !is.na(n)), aes(method, n, fill = class)) +
  geom_col(width = .65) +
  scale_fill_manual(values = c("healthy (0)" = cores_smf[["claro"]],
                              "diseased (1)" = cores_smf[["alerta"]])) +
  labs(x = NULL, y = "training rows", fill = NULL,
       title = "Class balancing: training only") +
  theme(legend.position = "top")

```

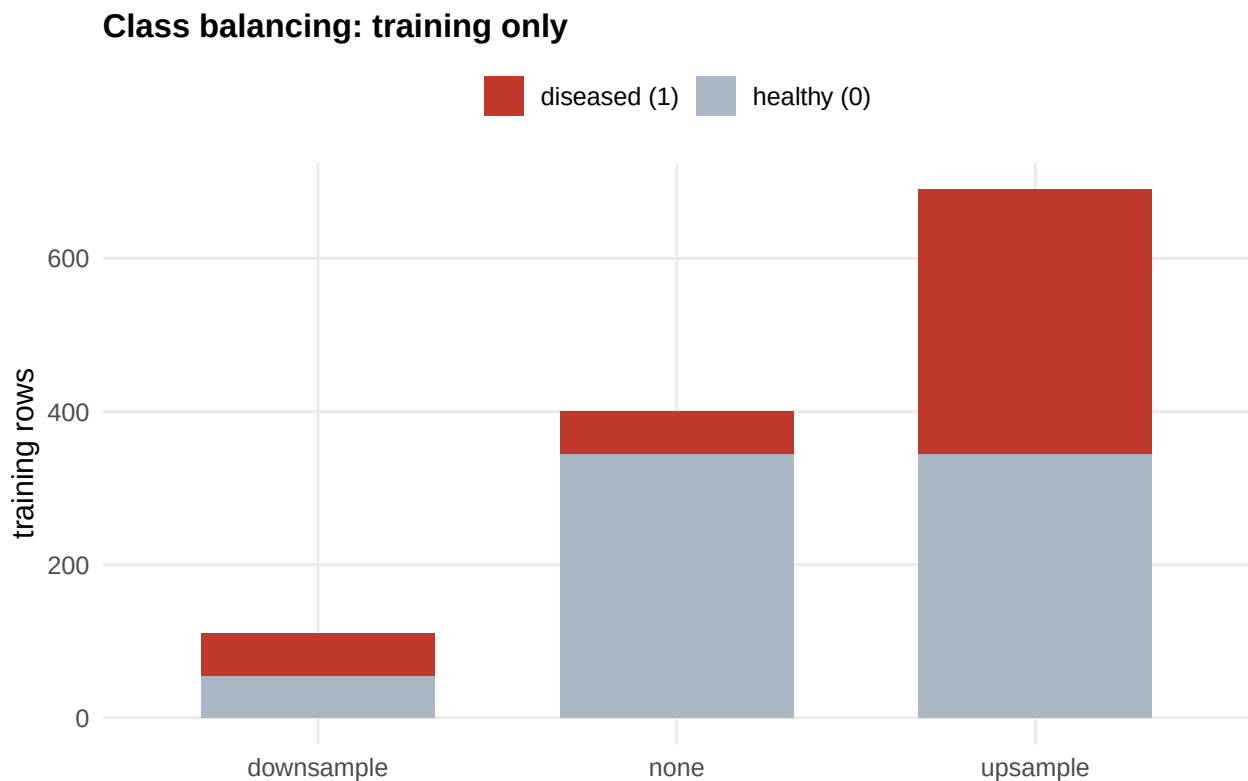


Figure 6.1: Balancing on training: downsample equalises classes by shrinking the majority; upsample equalises by replicating the minority; testing is never touched.

⚠ Trap

Balancing **before** splitting contaminates the test set: the same record can be duplicated in training and charged in testing. The package requires `context = "training"` and refuses `context = "test"` (seen in Part 1). SMOTE requires the outcome as a factor — the error returned by the package shows exactly that.

7 Feature selection and representation

Selecting features on the whole dataset is a subtle form of leakage. The correct path is to select **inside each fold**. The result carries per-fold scores and a stability measure.

```
cv_field <- smf_vfold_cv(no_out, v = 5L, id_column = "plot_id", seed = 9L)
sel <- smf_feature_select_resamples(
  no_out, "yield", smf_feature_spec("mutual_information", n_features = 3L),
  cv_field, id_column = "plot_id"
)
sel_tbl <- merge(sel@scores, sel@stability, by = "feature", all = TRUE)
tab_smf(sel_tbl, caption = "Mutual-information selection (3 features per fold) repeated over
```

Table 7.1: Mutual-information selection (3 features per fold) repeated over 5 folds: selection frequency and stability per feature.

feature	selection_frequency	frequency
block	0.0	0.0
genotype	0.0	0.0
nitrogen	1.0	1.0
organic_matter	0.6	0.6
potassium	0.6	0.6
water	0.8	0.8

```
stability_sel <- sel@stability
names(stability_sel)[2] <- "frequency"
ggplot(stability_sel, aes(stats::reorder(feature, frequency), frequency)) +
  geom_col(fill = cores_smf[["primaria"]], width = .6) +
  geom_text(aes(label = fmt(frequency, 2)), hjust = -0.15, size = 3.2) +
  coord_flip() +
  labs(x = NULL, y = "frequency across folds",
       title = "Feature selection stability across folds") +
  ylim(0, 1.12)
```

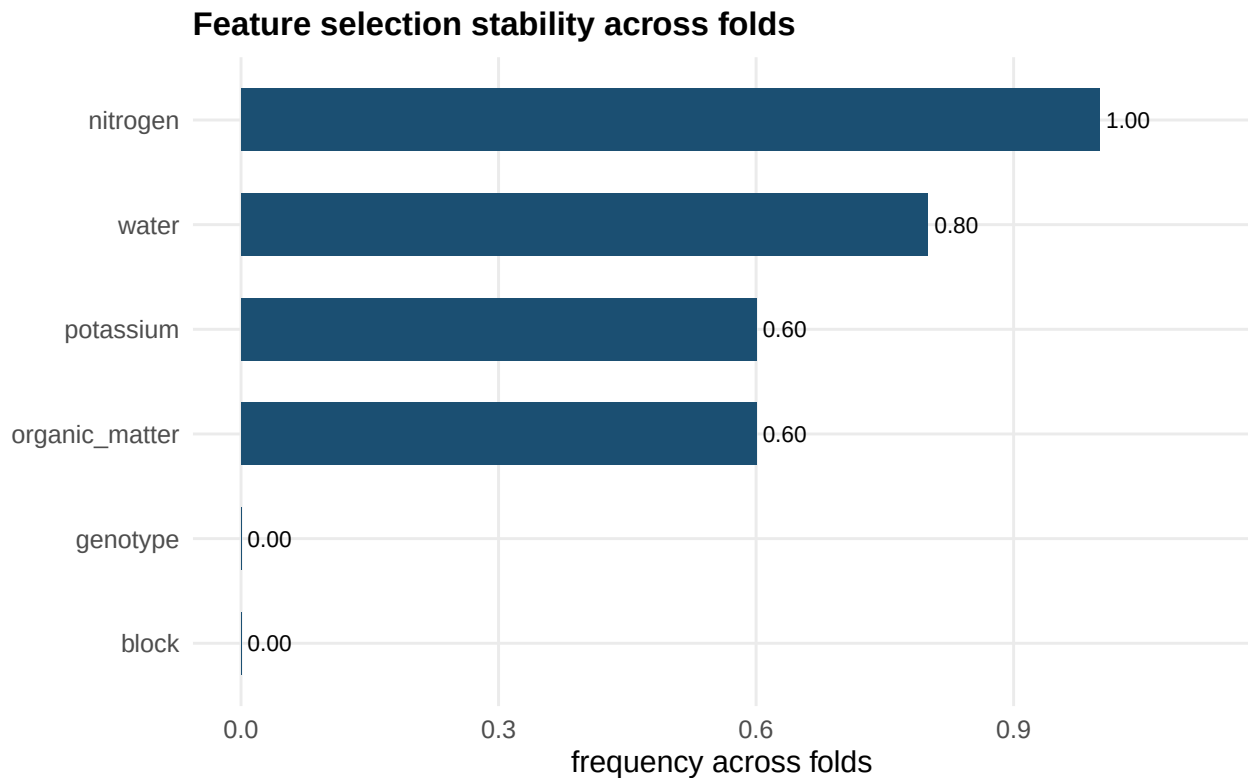


Figure 7.1: Selection stability across folds: features chosen in every fold are the reliable candidates.

In high dimension (200 wavelengths for 120 samples), the solution is **representation**: PCA or PLS reduce dimensionality before the model, always fitted on training. The experiment below uses the PCA representation declared in the specification.

```
spectral <- ler("spectral_pn")
waves <- grep("^w", names(spectral), value = TRUE)
make_rep <- function(k) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("regression", "protein"),
    data = smf_data_spec("protein", waves, id_column = "sample_id"),
    design = smf_design_spec(id_column = "sample_id"),
    features = smf_feature_spec("pca", n_features = k, representation = "pca"),
    resampling = smf_resampling_spec("holdout", train_prop = 0.75, seed = 4L),
    model = smf_model_spec("linear_regression", "stats"),
    metrics = list(smf_metric_spec("rmse")),
    reproducibility = smf_reproducibility_spec(4L)
  )
  smf_fit_experiment(spec, spectral)
}
fit_rep <- make_rep(8L)
var_exp <- cumsum(fit_rep@fit@features@state$sdev^2) /
  sum(fit_rep@fit@features@state$sdev^2)
```

```

tab_smf(data.frame(component = seq_along(var_exp),
                  cumulative_variance = var_exp),
        caption = "Cumulative variance of the principal components fitted on training.")

```

Table 7.2: Cumulative variance of the principal components fitted on training.

component	cumulative_variance
1	0.468
2	0.756
3	0.992
4	0.993
5	0.993
6	0.993
7	0.994
8	0.994
9	0.994
10	0.994
11	0.995
12	0.995
13	0.995
14	0.995
15	0.995
16	0.996
17	0.996
18	0.996
19	0.996
20	0.996
21	0.996
22	0.997
23	0.997
24	0.997
25	0.997
26	0.997
27	0.997
28	0.997
29	0.998
30	0.998
31	0.998
32	0.998
33	0.998
34	0.998
35	0.998
36	0.998
37	0.998
38	0.998
39	0.999
40	0.999

component	cumulative_variance
41	0.999
42	0.999
43	0.999
44	0.999
45	0.999
46	0.999
47	0.999
48	0.999
49	0.999
50	0.999
51	0.999
52	0.999
53	0.999
54	0.999
55	0.999
56	0.999
57	1.000
58	1.000
59	1.000
60	1.000
61	1.000
62	1.000
63	1.000
64	1.000
65	1.000
66	1.000
67	1.000
68	1.000
69	1.000
70	1.000
71	1.000
72	1.000
73	1.000
74	1.000
75	1.000
76	1.000
77	1.000
78	1.000
79	1.000
80	1.000
81	1.000
82	1.000
83	1.000
84	1.000
85	1.000
86	1.000
87	1.000

component	cumulative_variance
88	1.000
89	1.000
90	1.000

```
rmse_comp <- do.call(rbind, lapply(c(2L, 4L, 6L, 8L, 12L, 20L), function(k) {
  f <- make_rep(k)
  data.frame(components = k,
              rmse = f@metrics$value[f@metrics$metric == "rmse"])
}))
tab_smf(rmse_comp, caption = "Validation RMSE by number of principal components (true noise: 0.25)")
```

Table 7.3: Validation RMSE by number of principal components (true noise: 0.25).

components	rmse
2	0.946
4	0.306
6	0.307
8	0.307
12	0.302
20	0.331

```
p_var <- ggplot(data.frame(component = seq_along(var_exp), var = var_exp),
               aes(component, var)) +
  geom_line(colour = cores_smf[["primaria"]]) +
  geom_point(size = 1.6) +
  labs(x = "components", y = "cumulative variance",
       title = "PCA variance (training fold)")
p_rmse <- ggplot(rmse_comp, aes(components, rmse)) +
  geom_line(colour = cores_smf[["destaque"]]) +
  geom_point(size = 1.6) +
  geom_hline(yintercept = 0.25, linetype = "dashed", colour = cores_smf[["alerta"]]) +
  labs(x = "components", y = "RMSE (validation)",
       title = "RMSE vs components")
patchwork::wrap_plots(p_var, p_rmse)
```

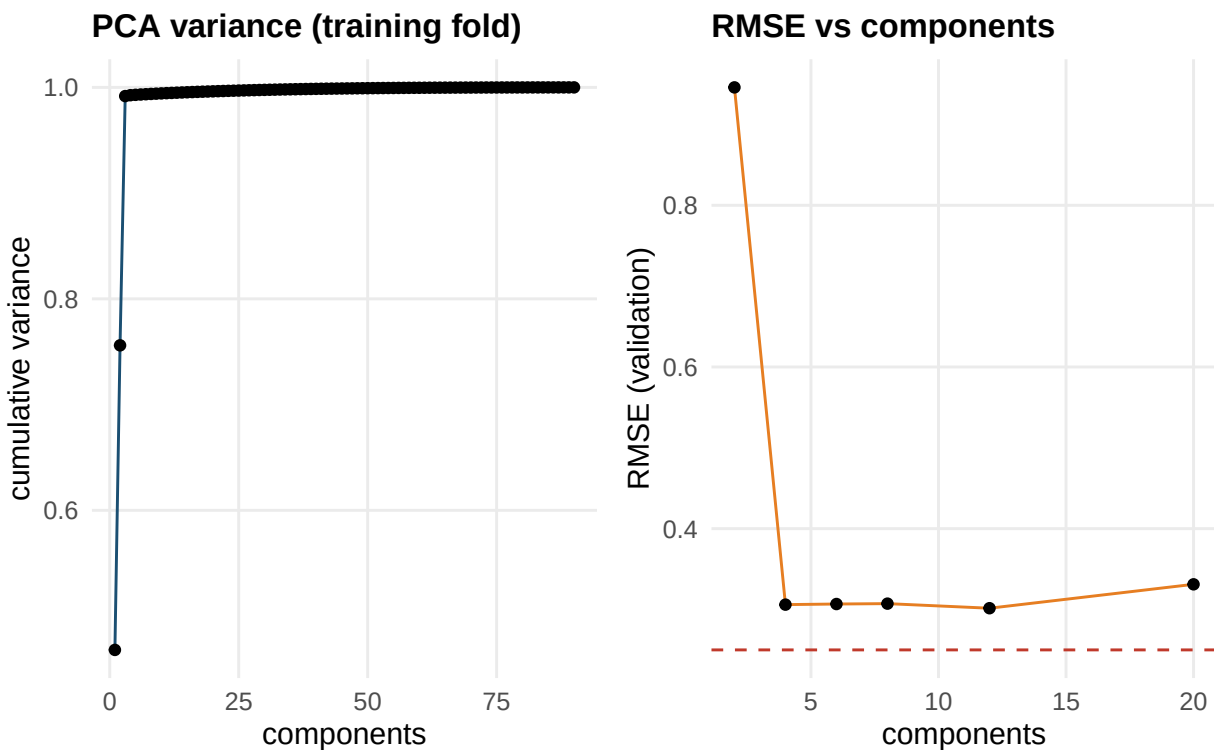


Figure 7.2: PCA representation: cumulative variance (left) and validation RMSE by number of components (right).

i Interpretation

With 120 samples and 200 wavelengths, the direct linear model is unstable; the representation solves it. Validation RMSE drops until 12 components (0.302) and then approaches the true noise (0.25). Keep this dataset: in Part 3 it compares PCA and PLS representations, and in Part 5 it feeds explainability.

8 Part 2 exercises

Exercise 2.1 — Prove the leakage

In ordinary v-fold on `farm_groups` (with `override = TRUE`), count how many farms appear on both sides of each fold. Then repeat with `group v-fold` and show the count is zero. (Hint: `s@train_index`, `s@test_index` and `farms$farm`.)

Exercise 2.2 — Verify the time order

In `smf_time_cv()` on `climate_series`, confirm that in **every** fold the test day comes after the last training day. Then redo it with `smf_vfold_cv()` and show a case where the order is violated.

Exercise 2.3 — When does leakage disappear?

Repeat the v-fold vs group v-fold comparison **without** the `farm` column in the predictors. Does the optimism persist? Explain why, and what this teaches about when the wrong scheme is harmless (and why it is still risky).

Exercise 2.4 — MCAR versus MAR in imputation

Compare the nitrogen coefficient (MCAR) and the organic-matter coefficient (MAR) under the three imputation strategies. Which coefficient is more sensitive? Relate it to the missingness pattern seen in Part 1.

Solutions

2.1. With forced ordinary v-fold, every fold has plots from farms that are also in training (overlap > 0 in all folds). In `group v-fold`, the train test farm intersection is zero by construction.

2.2. In `smf_time_cv()`, `max(train) < min(test)` in all folds (expansion into the future). In `smf_vfold_cv()`, since the split is random, some fold will have test days earlier than training days — the model “learns the future”.

2.3. Without `farm` in the predictors, the model cannot memorise the farm effect: the optimism essentially disappears. But it remains risky: it suffices that **some** attribute carries farm information (soil, local climate, management practice) for leakage to return. The right design does not depend on the model “not being able” to exploit leakage.

2.4. The organic-matter coefficient (MAR, block-dependent) is more sensitive: missingness concentrates in B4, and global-mean imputation ignores that structure. Nitrogen (MCAR) suffers less. The Part 1 diagnostic (missingness by block) already predicted this.